Partition-Tolerant Federation Across Emulated ISP Islands

Anonymous submission

***Abstract*—An Internet shutdown is rarely a clean power switch. Transit to the outside world can be withdrawn while domestic networks keep carrying packets among themselves, leaving reachable fragments that recent measurement work calls connectivity islands. We ask a deliberately narrow question: when some domestic IP paths survive, can independently operated forum servers keep accepting signed posts and keep reconciling them across whatever graph remains? Our prototype combines four mechanisms that are individually familiar and, to our knowledge, have not been composed for this operating regime: canonical protobuf envelopes admitted as the exact bytes their author signed, operator receipts bound to those bytes, caller-initiated synchronisation that survives asymmetric reachability, and an explicitly trusted dual-homed relay between islands. On a four-node testbed whose shared exchange segment is withdrawn mid-run, a dependent certificate, community, and post crossed the severed boundary in 0.5, 1.2, and 2.1 seconds—within a factor of two of the same chain measured while the exchange was intact. An eight-node chain delivered 200 of 200 envelopes seven hops with a 4.125 s median and a 6.115 s 99th percentile, and per-hop cost tracked a queueing model to within 89 ms across a fourfold change in the drain interval. TypeScript, Rust, and Python implementations agreed on all 16 canonical vectors. A signature-invalid flood produced no database writes, though a rate limiter shed 90% of it before the pipeline was reached, which we report rather than headline. Against a median gzipped ActivityPub activity our envelopes are 4.4–6.9× smaller, at the cost of Fediverse interoperability and JSON-LD extensibility. These results establish partition tolerance under surviving IP paths on a single-host testbed. They establish nothing about total blackouts, real inter-provider deployment, or resourced denial of service.**

***Index Terms*—federated systems, Internet shutdowns, network partitions, signed objects, store-and-forward**

I. Introduction

Internet shutdown doesn't necessarily mean a complete loss of connectivity. The disruption can occur in several forms like reduction in bandwidth, blocking destinations, filtering protocols, and loss of international routes. Despite these restrictions, domestic networks can still communicate with each other [1], [2]. Baltra et al. described the groups that lost the core internet connection but still can communicate locally as "islands" [3]. They also described networks that still have limited connection with the internet as "peninsulas" [3]. They have explained how these disrupted situations can be identified and developed measurements for them. Our work focuses on developing a system which can continue working in these crucial situations. Our system tries to answer a specific question: if the international routes are cut but domestic IP connection remains connected and a forum server works independently, whether they can receive signed posts and synchronize those posts across the network?

Other existing systems focus on a related problem which is not exactly similar to ours. Circumvention systems are designed to reach a blocked region. Telex depends on cooperation from network infrastructure to help a blocked user to reach outside the network. On the other hand, Snowflakes does this through proxy servers [4], [5]. In their cases, all of them have to reach either the network infrastructure or the proxy server. But in case of internet cut, they can reach neither them when all external paths are unavailable. Anix, Rangzen, Moby, and Twimight tried to construct a device-to-device or mobile mesh network for a situation where general internet access is not available [6]–[9]. They have used technologies such as Bluetooth or Wi-Fi, where messages pass from one device to another nearby device and the device relay the message farther after receiving it. Though they can continue to messaging while complete blackout, message delivery may take huge time due to network congestion, mobility, intermittent device etc. Also, they might waste additional power and resources of the device. So far we've studied, none of them have studied if independent servers can continue to check, store and relay messages maintaining a strong local network. Our research addresses this gap and tries to find a solution.

Our designed system follows a principal rule: author signs a fixed sequence of bytes, receiving server verifies it without changing any bytes. While forwarding an object through a relay, it can never be repaired, normalized or re-encoded. When a server accepts an object, it then issues a receipt which is cryptographically bound to object's content id and acceptance proof. It not only provides the object's content ID, but also proves that the server accepted the canonical content identified by that ID. Synchronization starts from the caller server, so the server which can't receive incoming connections, can also publish objects and restore missed objects from other servers. If two islands can't communicate directly, they pass messages through a trusted bridge server which is connected to both. The bridge server can verify, save and relay objects. Just like other servers, it does an admission check of the objects.

This paper makes mainly 4 contributions:

1) Server follows same byte preserving admission process for both locally saved object and object found from other servers. So, a protocol-compliant relay does not change a malformed object into a valid one.
2) It supports caller-initiated delivery, streaming and backfill. So, even if a server can't accept an incoming

connection, can still publish and recover objects by establishing an outbound connection to known peer.

3) It defines a quota bound relay policy. Relay verifies the object again, checks quota. Most importantly, it doesn't send the object back to the server again from which it has received it. A trusted peer may get larger quota and access to additional traffic classes while sending or receiving messages. But it doesn't mean that a post by truster peer is automatically valid.
4) It reports a result from a testbed where connection between islands were disconnected while experiment. The experiment also includes cross-language byte agreement tests, performance change with respect to increasing hop, flood of invalid signatures and a comparison with ActivityPub.

Our claim is limited to partition tolerance only when some IP paths remain between servers directly or through a bridge. This proposed designed cannot create a new route during complete blackout. This cannot force a server to accept a valid post or hide any visible relay from its hosting network operator.

II. Background and Research Gap

A. Federated and Signed System

ActivityPub directly delivers social activities to the server's inbox [10]. AT protocol supports federation and repository replication [11], [12]. Nostr and Secure Scuttlebutt replicates object signed by the author [13], [14]. They generally assume that some relevant server, relay, or peer endpoint remains reachable. But none of them has evaluated exact-byte admission, operator receipts, restoration through outbound connection and relay policy for connection between two network islands.

B. Communication During Blackout

Anix, Rangen, Moby and Twimlight establishes a network connecting nearby devices [6]–[9]. Dolphin uses cellular voice channel when normal packet is blocked [15]. These systems work without internal server connection. But they sacrifice bandwidth, devices extra power consumption and network capacity. Our system chooses the opposite trade-off: when the servers are connected, it establishes connection between them, but can't work when all IP paths are totally disconnected. Those systems don't compete with one another, rather they complement each other. So, this paper doesn't compare their latency, because they are designed for different workloads.

C. Accountability:

Append-only logs, signed tree heads and inclusion proofs has been created to audit certain kind of harmful behavior father by certificate transparency [16], [17]. PeerReview and CoSi explores several stronger forms of accountability by using witnesses and cosigning [18], [19]. Our receipt has been kept comparatively weaker intentionally. It only proves that an operator has acknowledged a specific content ID at a stated time. It doesn't prove that operator will receive valid object, will forward object faster or object will not be rejected silently.

D. Study gap and design objective

There is a difference currently present two fields. Normally routed federation assumes that server is always directly connected to the network. On the other hand, device to device assumes that network is totally disrupted. We've studied an intermediate situation between them where independent servers are working and there exists internal paths while the international routes are completely disconnected. In this situation, our goal is to admit each signed object once per server, keep its signed bytes unchanged, and allow every reachable server using the same rules and prerequisite state to reach the same validity decision.

III. System Design

A. Architecture

The system is designed around two premises: a forum should keep working as connectivity narrows, and should stress censorship resistance. We have denoted and modeled narrowing as a five-level ladder — L0, ordinary global (or Tor) reach; L1, national partition, where international transit is gone but the domestic exchange still carries traffic; L2, ISP island, where inter-ISP peering itself has failed; L3, bridged islands, where a dual-homed relay reconnects two islands that cannot otherwise reach each other; and L4, total isolation, where no IP path is available.

The ingress is made polymorphic and heterogeneous to match this ladder : HTTP/gRPC is used over whatever IP path survives, a Tor v3 onion service is used when an external Tor path persists (and server / user wants to use that), and an optional Reticulum/LoRa sidecar carrying only small, high-priority messages when IP is unavailable altogether. Bulk traffic is rejected on that path, and it is disabled by default. That sidecar is our only answer to L4, and it is a narrow one: this paper focues solely the IP-networking path (HTTP for clients, gRPC for federation), and its claims are scoped to L0 through L3.

Edited version:

The canonical signed envelope is invariant across ingress media and relay paths. The transport context and admission context are not. Every node validates the envelope through the same 19-stage admission pipeline. The first 12 stages validate the envelope without persistent writes. If the envelope is accepted, the node commits it, signs a receipt, and places a federation task in a durable outbox. The scheduler drains the outbox by priority class, then by '(destination, source uplink, traffic class)'.

The node must still discriminate how the envelope arrived. Let C_1 denote a client, and let S_1 and S_2 denote independent forum servers. In the illustrative case $C_1 \rightarrow S_1$, the client submits directly to its chosen server. In the illustrative case $C_1 \rightarrow S_1 \rightarrow S_2$, server S_1 relays the same signed envelope to server S_2. These are examples, not an exhaustive enumeration. The canonical bytes, content identifier, and author signature remain unchanged. What changes is the admission context: ingress kind, supplying peer, direction, source uplink, traffic

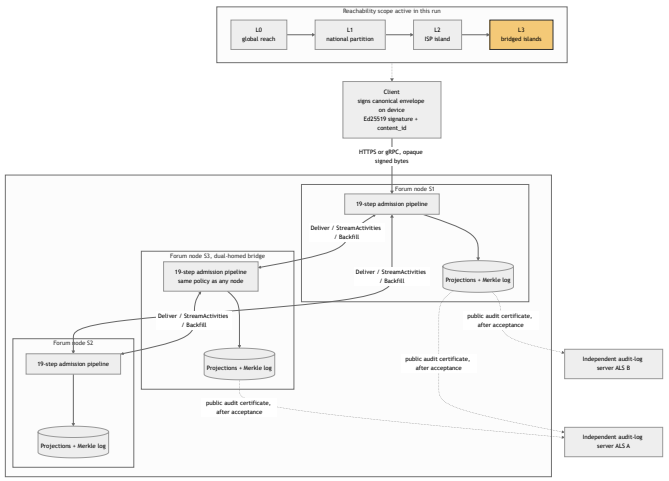

Fig. 1. System architecture. Clients sign canonical envelopes on-device and submit them to forum nodes over HTTPS or gRPC; every node runs the same 19-step admission pipeline, persists accepted envelopes into projections and a Merkle log, and exchanges exact signed bytes with peers via Deliver / StreamActivities / Backfill. Independent audit-log servers (ALS A, ALS B) receive public audit certificates after acceptance. The dual-homed node S3 acts as a bridge under the same admission policy. The reachability scope at the top indicates which paths are usable at a given moment.

class, and quota policy. That context controls deduplication, replay protection, scheduling, and fan-out. It never relaxes signature, identifier, or body validation.

A bridge is an ordinary node running the same stack, dual-homed onto two islands; but not a transparent packet forwarder. It's all relaying capability has already been independently validated, stored, and quota-bounded under its own key.

Independent audit-log services (ALSs) provide an external proof of server acknowledgement. This holds the server owners accoutntable. They operate outside the write path. After a server commits the write it returns a signed receipt. The client builds a public audit certificate, stores it locally, and then forwards it to one or more ALSs. Each ALS independently re-verifies the canonical envelope, content identifier, receipt, signed tree head, and inclusion proof before appending the certificate to its own hash-chained record. If any server later hides or deletes the accepted object, the acknowledgement remains externally verifiable. This supports censorship resistance by limiting a single server's ability to silently erase evidence of acceptance. Fig. 1 shows the system end to end: the reachability ladder, the on-device signing boundary, the canonical envelope, the shared admission pipeline, each node's persistence stack, and the independent audit-log services.

B. Signed objects and admission

An envelope carries a domain tag, an object type, a payload, a public key, a nonce, and an expiry. Its identifier is the SHA-256 digest of a specified canonical byte string, and an Ed25519 signature covers that same string [20]. Protobuf does not guarantee a unique serialisation [21], so we constrain

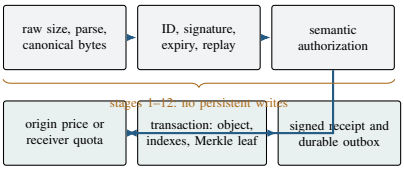

Fig. 2. Admission pipeline. State changes begin only after the cheap byte and validity checks have all succeeded.

a profile on top of it — ascending field order, minimal varints, no unknown fields retained — and add the rule that actually matters: decoding and re-encoding an object must reproduce the bytes as received, exactly. An object that fails that comparison is rejected outright, never repaired.

Verifying the exact received bytes after a strict decode is established practice in DER, JWS, and Certificate Transparency leaf formats. In this system, the rule protects federation. If a relay decoded a peer's bytes and re-encoded them before checking the signature, it could normalise a non-canonical submission into the canonical form that was signed. The relay would then verify reconstructed bytes rather than the bytes that arrived. Our decode–re-encode comparison prevents a receiver from repairing or normalising attacker-controlled bytes before verification. As a result, no relay can make a non-canonical or invalid submission acceptable by rewriting it.

Fig. 2 lays out the admission path. Stages 1 through 12 — size, parsing, canonicality, domain separation, identifier derivation, signature, expiry, replay, authorisation — write nothing to persistent storage, so an invalid flood cannot amplify into write load, a property we measure directly in §V. The stages after that charge origin pricing or a receiver quota, commit the raw object with its derived indexes in one transaction, append its hash to a Merkle log, and only then enqueue federation. Exact retries are made idempotent under concurrency by a uniqueness constraint in the database, not by a read-then-write check that a race could defeat.

On acceptance, the server signs a receipt binding the content identifier, the leaf position, the resulting signed tree head, and an inclusion proof. The acknowledgement therefore commits to the exact bytes admitted. Peers also exchange signed tree heads containing a tree size, root hash, and timestamp. Two heads of equal size with different roots are an unambiguous conflict. A larger head is recorded as newer, but it is treated as proven only after consistency proof is obtained. A smaller head is also not immediately treated as a fork, as observations may arrive out of order: in case the timestamp is older than the recorded head, it is discarded as stale; otherwise, a smaller size is reported as a possible fork. This is deliberate stale-head handling. It removes a class of self-inflicted false positives and catches outright tree shrinkage, however it is not full fork detection: A adversary who rewrites history while keeping the tree size non-decreasing can pass these local checks, and equivocation between two peers cannot be detected by a single local comparison. Detecting those cases requires consistency proofs and independent cross-observer exchange, which the

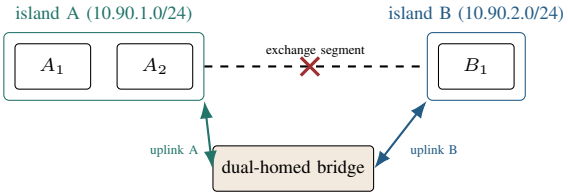

Fig. 3. Route-cut topology. Four application nodes span two isolated network segments joined by a shared exchange, which the experiment withdraws mid-run.

network partitions we target are likely to obstruct.

C. Partition-aware federation

Three caller-initiated RPCs cover ordinary delivery and recovery. DELIVER pushes a batch of accepted envelopes to a peer. STREAMACTIVITIES resumes a live stream from a durable cursor. BACKFILL requests objects that the node knows it missed, using a resumable log position. Because all three operations are initiated by the caller, a node does not need to accept inbound federation connections. A server behind carrier-grade NAT can therefore federate in both directions over connections it opens itself. This solves reachability asymmetry, not discovery: at least one peer address must already be known.

The scheduler keeps an independent durable queue for each destination peer. It drains these queues concurrently under bounded leases, with deadlines, exponential backoff, and idempotency keys. This structure exists because its absence was a real defect, described in §V: a serial drain allowed one unreachable peer to block delivery to every other peer, including the bridge. One caveat belongs here rather than in the limitations section. A delivery deadline bounds how long the drain pass waits on a peer; it does not cancel the underlying transport call. Because the sender holds no cancellation token, a stuck call can continue to hold resources after its deadline, even though it no longer blocks delivery to other peers.

Fig. 3 shows the route-cut topology used in the evaluation. Island A contains two ordinary nodes, island B contains one, and a shared exchange segment stands in for an Internet exchange point. The bridge holds one address on each island network. Once the exchange segment is withdrawn, both islands must already have configured trust in the bridge. That trust determines the rate and priority of the bridge's traffic; it never determines whether an envelope is valid. The bridge validates and stores every object it relays, enforces quota per uplink pair and traffic class, and never sends an object back to the peer from which it arrived.

Four implementation rules carry most of the replication safety. Peer bytes are passed through unchanged: the transport passes received envelopes as opaque byte strings and does not decode or re-encode them. Trust controls quota and priority, never validity; every peer, including the bridge, still repeats the full admission checks. Duplicate delivery is inevitable when a live stream and a backfill race, and it is absorbed by a database uniqueness constraint rather than by a read-then-write check. Fan-out excludes the peer that supplied an object, avoiding a redundant round trip that deduplication would eventually terminate, but not for free.

IV. METHODOLOGY

We evaluate five questions: whether independent encoders agree on the canonical form, whether objects cross an enforced route cut, how delivery latency scales with hop count, whether malformed requests produce storage writes, and how the representation compares in size with ActivityPub. All experiments run on one physical host; §VI treats the consequences of that choice at length.

Correctness tests. A shared corpus of 16 positive and adversarial byte vectors is executed independently by the TypeScript, Rust, and Python implementations. Each must agree byte-for-byte on encoding and identifier derivation, and each must reject every negative vector. A further 85 focused tests cover ingress (29), the outbox (3), federation (31), and simulated ISP behaviour (22). These are regression tests written by the system's authors, not an independent security audit.

Route-cut experiment. Four application nodes, two databases, and two caches are distributed across three isolated virtual network segments, as shown in Fig. 3. Containers supply reproducible layer-2 isolation and let the harness withdraw the only configured inter-island path mid-run; nothing in the design depends on that packaging.

The test proceeds across four sequential stages. First, the harness inspects kernel TCP state via `/proc/net/tcp` inside the bridge to verify the exact local source address from which each established federation connection departed. Second, it withdraws the shared exchange segment and validates that direct inter-island probes between island A and island B consistently fail. Third, the test verifies that the destination island does not already contain the content scheduled for submission. Fourth, it pushes a dependency chain (an author certificate, a community record, and the dependent post) and polls for exact content identifier matches on the opposite island. Because a post cannot be rendered without its accompanying author certificate and community metadata, the recorded crossing latency accounts for all three artifacts rather than merely the terminal post. Altogether, 19 assertions validate network topology, source interface binding, relay forwarding semantics, quota allocation, and uplink failover policies. Because reported latencies incorporate the application's internal polling intervals alongside transit delay, they represent conservative upper bounds on actual delivery time.

Hop-scaling chain. Eight nodes are peered into a line, each with its own database, so that a node i hops from the origin is reachable by exactly one path of exactly that length. Hop count is the independent variable. A mesh would destroy this property, since in a mesh every node is one hop away and only fan-out width varies.

Latency is computed by subtracting the `received_at_ms` value each node writes inside the same transaction as its projection. No polling observer sits in

the measurement path. This is sound only because all nodes share the host clock, which a real deployment would have to replace with NTP discipline and an error bar. We inject 200 valid envelopes at hop 0 at a paced rate and record arrival at hop 7. The eight nodes share a single database process with one database each, which introduces contention and inflates the reported figures—the conservative direction in which to be wrong.

Signature-invalid flood. Random bytes are refused within microseconds at the parse stage and measure nothing interesting. The tool therefore lifts a genuine envelope from the node's own log and mutates a single byte inside the signed region. This invalidates the signature and changes the content identifier, so every request is novel and none can be short-circuited by deduplication. We issue 3,000 such requests at concurrency 16 and record completion rate, rejection stage, the fraction reaching signature verification, and—via the database's collection-level profiler—the resulting writes. The ordinary rate limiter stays enabled, so this is not a saturation benchmark.

Encoding comparison. We collected 80 real `Create`/`Note` activities from the public outboxes of two stock ActivityPub deployments. Because a collection hoists `@context` to the collection while an activity POSTed to a remote inbox carries its own, measuring the bare outbox item would understate delivery size. The tool therefore refetches a standalone object from the same host and reattaches exactly those bytes. We compare raw and gzipped activity bytes against equivalent signed envelopes, subtracting authored UTF-8 text so the metric reflects what a protocol designer controls rather than how much a user typed.

The comparison is deliberately generous to ActivityPub. It excludes HTTP framing, TLS, per-follower fan-out, shared dictionaries, and—most significantly—authentication, since our bytes carry a 64-byte signature verifiable at rest while HTTP Signatures are transport level and vanish with the request.

V. Results

A. Correctness and convergence

Every implementations agreed on all the 16 canonical vectors and rejected all the negative ones as specified. The additional tests also checked invalid signatures, incorrect byte formats, replay attacks, missing permissions, duplicate messages, wrong signing keys, invalid proofs, outdated tree heads, and conflicting logs. Sixteen vectors cannot cover every protobuf's edge cases. However, they show that those three independently developed implementations follow exactly the same byte-level rules. This is essential for a relay network to work reliably.

The route-cut test passes all 19 checks. The TCP connection check also confirmed that the federation connections used both configured addresses, 10.90.1.30 and 10.90.2.30. But this could only be verified with real network sockets, not an in-process test. When direct communication between the islands was disabled, Island B still received the certificate

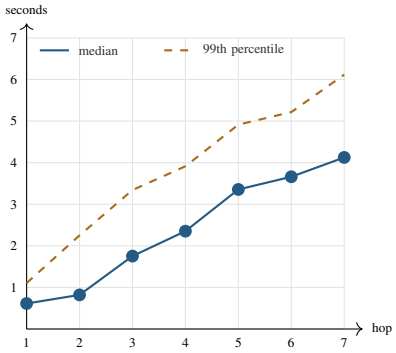

Fig. 4. Measured per-hop latency across the eight-node chain at a 1,000 ms drain interval, from in-transaction timestamps. All 200 envelopes reached hop 7.

after 0.5 s, the community after 1.2 s, and the post after 2.1 s. This follows their dependency order through the bridge. For comparison, with the direct connection still available, the same items took 1.5 s, 1.5 s, and 1.9 s. So the bridge worked correctly and was still within about twice the speed of the direct connection.These are results from single test runs, and not statistical averages.

The 2.1 s result was achieved through a specific fix and was not just simply observed. An earlier version passed 18 of 19 tests, but the one successful crossing took 407.6 s, with all outbox records still showing zero delivery attempts. The problem was that peers were contacted sequentially with no time limit. If one peer was unreachable, it blocked delivery to all the others. This includes the bridge that was supposed to bypass the network failure. We were able to fix this by delivering to peers concurrently and adding a time limit to each delivery. This reduced the crossing time from 407.6 s to 2.1 s, which is a 195× improvement.

B. Latency scaling and admission cost

All 200 envelopes successfully traveled through the eight-node chain to hop 7. The median End-to-end latency was 4.125 s and the 99th percentile 6.115 s. The per hop results which are in Fig. 4 are more useful than the final numbers. Most of the delay comes from the outbox drain interval, not the protocol itself, which if predicted roughly then it's $drain/2$ plus a fixed term; With a 1,000 ms drain interval, each hop took about 589 ms, including roughly an 89 ms fixed protocol work. With a 250 ms interval, each hop took about 196 ms, implying 71 ms of fixed work. Since this fixed cost stayed around 71–89 ms even when the drain interval changed, it actually represents the protocol's actual per-hop cost. This includes verifying, projecting, appending, and re-queuing the message. The remaining delay is mainly a scheduling choice that operators can adjust based on bandwidth and battery needs. However, these results should not be treated as Internet performance predictions because the test used a shared database process and did not include real wide-area network delays.

The invalid-signature flood was rejected at 730 requests per second. This was all done with no writes to objects,

indexes, logs, or the outbox. The test also confirmed that the profiler was actually running as it recorded six read operations during that test. Of the 3,000 total requests, about 2,700 were blocked by the per-peer rate limiter. So, only around 9% reached signature verification, and then the expensive rejection process, which is the canonical parsing followed by Ed25519 verification. This ran about 66 requests per seconds. The final conclusion is a bit limited but clear enough. Invalid traffic was stopped before causing any writes, with the rate limiter handling most of the attack first. This test does not necessarily show how the system would handle floods from many different peers, valid messages that should cause writes, or long-term parser and database pressure.

Table I compares two encoding formats and their costs. Both tested hosts produced the same 927-byte `@context` declaration, which implies this is a fixed cost of the standard ActivityPub format, not something specific to our deployment. The declaration alone is more than four times larger than our entire signed forum post. Compression removes much of the apparent size advantage, so we use compressed ActivityPub numbers for a fair comparison. Gzipped ActivityPub messages were 786–1,400 B with a median of 1,074 B. Our raw signed envelopes were only 155 B, 220 B, and 243 B making ActivityPub about 4.4–6.9× larger. Our envelopes do not benefit from compression at all as the 64-byte Ed25519 signature is incompressible and gzip makes all three slightly *larger*. So the honest comparison is gzipped ActivityPub against our raw bytes, not the roughly 20× ratio suggested by comparing uncompressed sizes. However, size is the only area where our format is better. It cannot connect to the existing Fediverse, requires generated codecs and unfamiliar tools, does not allow unknown fields, and therefore requires versioned schemas for future changes.

VI. Discussion and Limitations

Testbed validity. All experiments were run on a single physical machine using virtual networks in Docker. This is the paper's main limitation.All nodes shared the same kernel, clock, CPU, and storage. The shared clock made it possible to compare timestamps across nodes, but real deployments do not have this advantage. The virtual networks were also much simpler than real networks, which involve physical routers, BGP changes, NAT, DNS failures, different MTUs, and wireless connections.Similarly, connecting a bridge to two virtual networks is easy, while a real deployment would require separate network links, routing rules, credentials, power, and an operator. Therefore, the route-cut experiment shows that the software can react correctly to a change in network and routing state. It does not demonstrate real geographic latency, independent failure domains, or real-world provider behaviour.

Access and subscriber isolation. The system assumes that nodes on the same network can communicate with each other or reach a shared bridge. However, some Wi-Fi networks and ISPs block direct communication between customers while still allowing them to access the provider's gateway. If this restriction exists on every available path, nodes cannot connect to each other or reach the bridge. In that situation, federation completely stops, and only local services continue to work.The current prototype has no alternative non-IP method for communication in this situation.

Network adversary. A network operator can do multiple things. They range from find stable endpoints, block IP addresses or SNI values, identify the RPC traffic, track communication timing, isolate a node to pressuring a bridge operator. Source-address binding only chooses which network connection to use; it does not hide the traffic.There is also one important failure case: **protocol whitelisting**. In this situation, normal domestic Internet routing still works, but deep packet inspection allows only approved protocols. Since the federation uses a distinctive, non-standard protocol on a specific port, it could be blocked immediately, regardless of how strong our signatures are. Possible solutions include running federation over normal HTTPS, changing endpoints regularly, or using pluggable transports. However, none of these solutions has been implemented or tested in this work.

System and application limits. Signatures prove who published something, but they do not prove that the content is truthful or safe. A legitimate key can still be used to publish harmful content or generate a large amount of traffic. Because the cost of admission is paid at the origin, an origin operator can create unlimited free proofs for its own users. Therefore, protection against abuse is only as strong as the weakest origin in the peer group. Per-peer limits are the main protection. Other problems remain open, such as stolen keys, exposed metadata, large-scale moderation, recovering from revoked keys, running out of storage, and coordinated fake identities (Sybil attacks). Transport timeouts only limit how long the system waits; they do not actually stop work that has already started. The system also specifies consistency proofs for growing trees, but these have not yet been implemented. Most importantly, a receipt only proves what the system **accepted and acknowledged**. If an operator simply refuses to accept something, no receipt is created. So, the simplest form of censorship which is silence or refusal to admit content, is impossible for this system to prove.

Evaluation limits. The route-cut results come from single test runs using one configuration. The encoding comparison used 80 activities of one type from two deployments. The system wasn't tested across multiple physical hosts, real wide-area networks, several days, or actual shutdowns. Host load, results from repeated runs, or confidence intervals weren't recorded. The resource measurements were taken on x86 computers, so they may not represent how the system would perform on the low-cost single-board computers that communities might actually use. Finally, the comparisons with proximity-based and voice-channel systems involve different types of workloads. They are meant for context, not to rank one system against another.

Ethics. All tests used fake content on isolated private networks, so no real users or sensitive data were involved. However, a real deployment could put bridge operators and users at legal or physical risk because a visible relay can be

TABLE I

MEASURED REPRESENTATION SIZE AND DESIGN TRADE-OFFS AGAINST ACTIVITYPUB. THE BYTE SAMPLE IS 80 REAL CREATE/NOTE ACTIVITIES FROM TWO STOCK DEPLOYMENTS. RATIOS COMPARE GZIPPED ACTIVITYPUB AGAINST RAW SIGNED ENVELOPES, WHICH DO NOT COMPRESS.

Dimension	This work	ActivityPub comparison	Cost or boundary
Measured bytes	Raw signed envelopes: check-in 155 B, forum post 220 B, broadcast 243 B; gzip enlarges all three	Delivered JSON-LD median 4,216 B, gzipped median 1,074 B (range 786–1,400 B); `@context` alone a fixed 927 B	Ratio 4.4–6.9×; representation only—HTTP, TLS, fan-out, shared dictionaries, and AP authentication all excluded in AP's favour
Interoperability	Custom protobuf domains and RPCs	Standard actor, object, and inbox model	No Fediverse compatibility and no existing application ecosystem
Evolution	Unknown fields rejected; versioned profiles required	Flexible JSON-LD vocabulary and open extension	Harder rolling upgrades and more cross-language test surface
Operations	Generated codecs and canonical-byte tooling required	Text inspectable with ordinary web tools	Higher debugging and tooling burden on operators
Integrity profile	One exact byte string per object, content ID, author signature, and receipt	Authentication depends on deployment profile; HTTP Signatures do not survive the request	Unambiguous byte identity, but a new security profile to maintain and audit

identified by the network hosting it. Therefore, the system should not be deployed in a hostile environment without first assessing local threats, getting informed consent, minimizing collected data, and reviewing the deployment with the affected community.

VII. CONCLUSION

Server federation can continue working after a route is cut, as long as domestic IP connections are still available. The system uses signed objects, one admission process for both local and federated traffic, caller-controlled recovery, and a trusted relay. Together, these allowed a dependent post to cross the broken connection in 2.1 s , which was within twice the time of the normal direct connection. The problem that caused an earlier 407.6 s delay only became visible when testing with three peers, including one unreachable peer. Tests across different programming languages also showed that they produced identical data, while the multi-hop test showed that the per-hop delay closely followed the expected queueing model, with only about 89 ms of additional cost. The compact format is 4.4–6.9× smaller than gzipped ActivityPubbut this comes at the cost of interoperability, extensibility, and mature tools. The most important future work is testing on multiple physical hosts and real wide-area networks, adding consistency proofs for growing logs, making transport cancellation-aware, testing floods of validly signed traffic, using multiple independent bridges, hiding the protocol from systems that allow only approved protocols, and providing a non-IP fallback for subscriber isolation and complete network outages.

REFERENCES

[1] Z. S. Bischof, K. Pitcher, E. Carisimo, A. Meng, R. B. Nunes, R. Padmanabhan, M. E. Roberts, A. C. Snoeren, and A. Dainotti, "Destination unreachable: Characterizing internet outages and shutdowns," in *Proceedings of the ACM SIGCOMM 2023 Conference*, 2023.

[2] T. I. Raso, S. Afrin, M. A. Chowdhury, M. Xynou, and E. Yachmeneva, "The longest silence: Internet shutdowns during Bangladesh's 2024 uprising," Digitally Right and Open Observatory of Network Interference, 2025, https://ooni.org/post/2025-bangladesh-report/.

[3] G. Baltra, T. Saluja, Y. Pradkin, and J. Heidemann, "Reasoning about internet connectivity," 2024.

[4] E. Wustrow, S. Wolchok, I. Goldberg, and J. A. Halderman, "Telex: Anticensorship in the network infrastructure," in *Proceedings of the 20th USENIX Security Symposium*, 2011.

[5] C. Bocovich, A. Breault, D. Fifield, Serene, and X. Wang, "Snowflake, a censorship circumvention system using temporary WebRTC proxies," in *Proceedings of the 33rd USENIX Security Symposium*, 2024.

[6] S. Kamali and D. Barradas, "Anix: Anonymous blackout-resistant microblogging with message endorsing," in *2025 IEEE Symposium on Security and Privacy*, 2025, pp. 1381–1399.

[7] A. Lerner, G. Fanti, Y. Ben-David, J. Garcia, P. Schmitt, and B. Raghavan, "Rangzen: Anonymously getting the word out in a blackout," 2016.

[8] A. Pradeep, H. Javaid, R. Williams, A. Rault, D. Choffnes, S. Le Blond, and B. Ford, "Moby: A blackout-resistant anonymity network for mobile devices," *Proceedings on Privacy Enhancing Technologies*, vol. 2022, no. 3, pp. 247–267, 2022.

[9] T. Hossmann, P. Carta, D. Schatzmann, F. Legendre, P. Gunningberg, and C. Rohner, "Twitter in disaster mode: Security architecture," in *Proceedings of the First ACM Workshop on Privacy and Security in Mobile Systems*, 2011.

[10] C. L. Webber, J. Tallon, E. Shepherd, A. Guy, and E. Prodromou, "ActivityPub," W3C Recommendation, 2018, https://www.w3.org/TR/activitypub/.

[11] The Matrix.org Foundation, "Matrix specification, version 1.11," 2024, https://spec.matrix.org/v1.11/.

[12] Bluesky Social, "AT Protocol repository specification," 2026, https://atproto.com/specs/repository.

[13] fiatjaf, "NIP-01: Basic protocol flow description," 2023, https://github.com/nostr-protocol/nips/blob/master/01.md.

[14] D. Tarr, E. Lavoie, A. Meyer, and C. Tschudin, "Secure scuttlebutt: An identity-centric protocol for subjective and decentralized applications," in *Proceedings of the 6th ACM Conference on Information-Centric Networking (ICN)*, 2019, pp. 1–11.

[15] P. K. Sharma, R. Sharma, K. Singh, M. Maity, and S. Chakravarty, "Dolphin: A cellular voice based internet shutdown resistance system," *Proceedings on Privacy Enhancing Technologies*, vol. 2023, no. 1, pp. 589–607, 2023.

[16] B. Laurie, A. Langley, and E. Kasper, "Certificate transparency," Internet Engineering Task Force, Tech. Rep. RFC 6962, 2013.

[17] B. Laurie, E. Messeri, and R. Stradling, "Certificate transparency version 2.0," Internet Engineering Task Force, Tech. Rep. RFC 9162, 2021.

[18] A. Haeberlen, P. Kouznetsov, and P. Druschel, "PeerReview: Practical accountability for distributed systems," in *Proceedings of the 21st ACM Symposium on Operating Systems Principles*, 2007, pp. 175–188.
[19] E. Syta, I. Tamas, D. Visher, D. I. Wolinsky, P. Jovanovic, L. Gasser, N. Gailly, I. Khoffi, and B. Ford, "Keeping authorities honest or bust with decentralized witness cosigning," in *2016 IEEE Symposium on Security and Privacy*, 2016, pp. 526–545.
[20] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, "High-speed high-security signatures," *Journal of Cryptographic Engineering*, vol. 2, no. 2, pp. 77–89, 2012.
[21] Google, "Protobuf serialization is not canonical," 2024, https://protobuf.dev/programming-guides/serialization-not-canonical/.